

Árvores de Derivação — Linguagem MicroJava

Construídas de acordo com a gramática formal do *MicroJava Quick Reference* (H. Mössenböck).

Convenções utilizadas nas árvores

<i>Símbolo em azul (itálico)</i>	Não-terminal da gramática (ex.: <i>Statement</i> , <i>Expr</i> , <i>Term</i>)
“símbolo” em preto (negrito)	Terminal literal: palavra reservada, operador ou pontuação (ex.: = , while , ;)
<i>símbolo em verde</i>	Classe terminal léxica: <i>ident</i> , <i>number</i> , <i>charConst</i>
<i>texto em laranja (tracejado)</i>	Lexema concreto reconhecido pelo analisador léxico (ex.: <i>sum</i> , <i>10</i> , <i>'A'</i>)

Gramática de referência (trecho relevante)

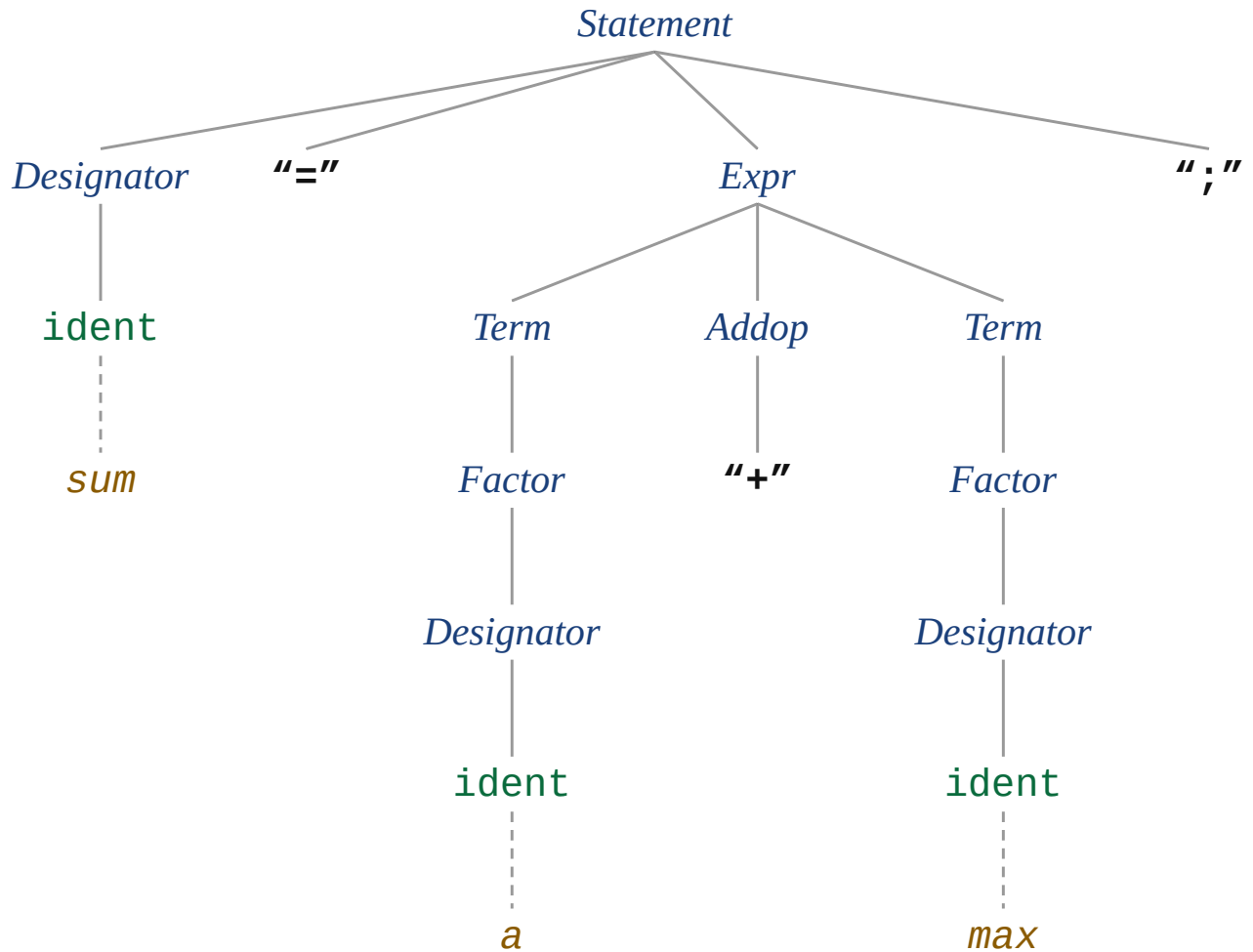
```
Statement = Designator ("=" Expr | ActPars) ";"
           | "if" "(" Condition ")" Statement ["else" Statement]
           | "while" "(" Condition ")" Statement
           | "return" [Expr] ";" | "read" "(" Designator ")" ";"
           | "print" "(" Expr ["," number] ")" ";" | Block | ";".

ClassDecl = "class" ident "{" {VarDecl} ";".
VarDecl   = Type ident "{" ident ";"".
Type      = ident "[" "[" "]"".
Condition = Expr Relop Expr.
Relop     = "==" | "!=" | ">" | ">=" | "<" | "<=".
Expr      = ["-"] Term {Addop Term}.
Term      = Factor {Mulop Factor}.
Factor    = Designator [ActPars] | number | charConst
           | "new" ident "[" [" Expr "]" ] | "(" Expr ")".
Designator = ident {"." ident | "[" Expr "]"}.
Addop      = "+" | "-".      Mulop = "*" | "/" | "%".
```

Observações: (1) em MicroJava, *int* e *char* são nomes de tipos pré-declarados e são reconhecidos pelo analisador léxico como *ident* (não constam na lista de palavras reservadas do guia de referência); (2) as partes opcionais [] e repetitivas { } da EBNF aparecem nas árvores apenas quando efetivamente utilizadas na derivação.

Declaração 1 **sum = a + max;**

Produção inicial: Statement = Designator "=" Expr ";"

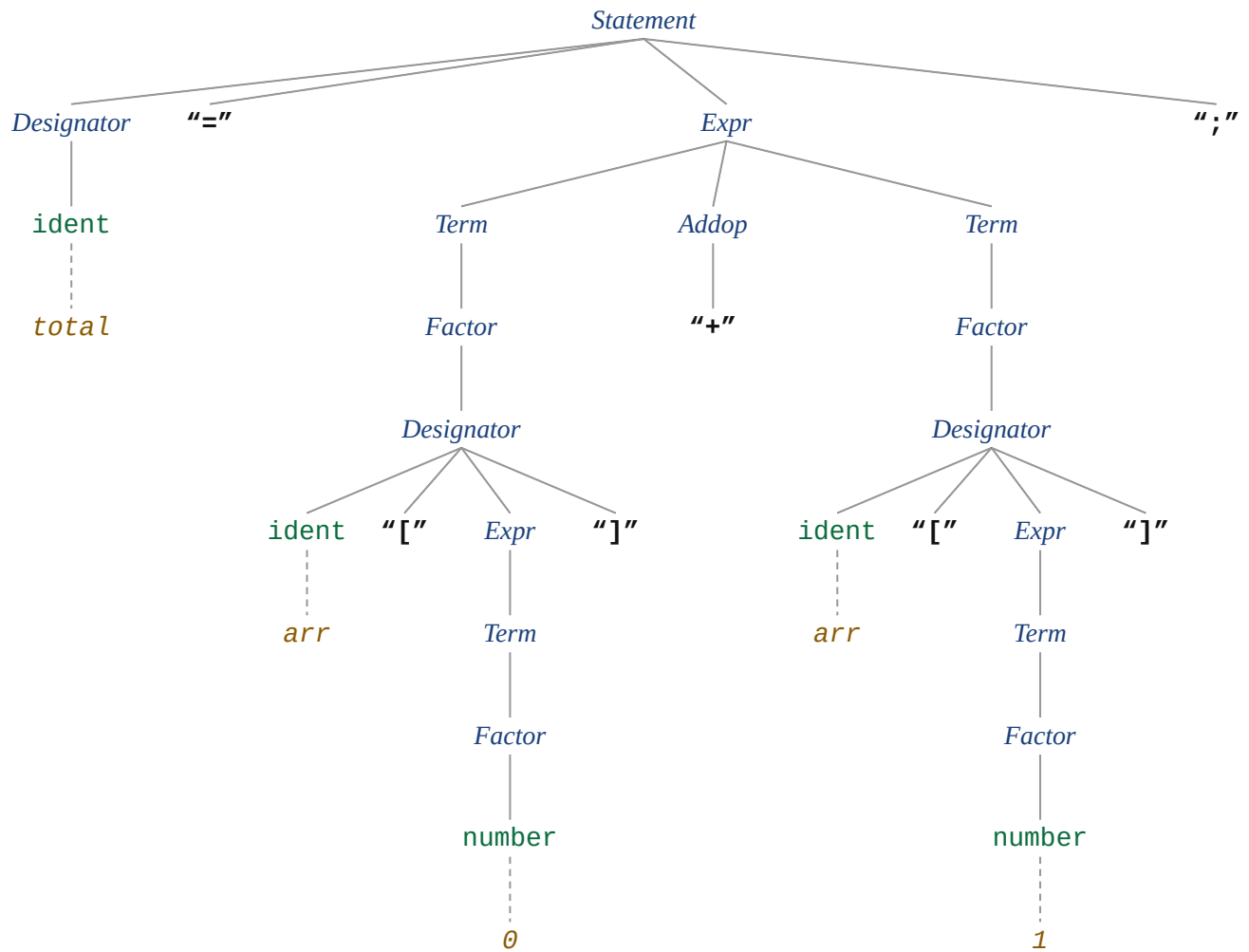


Produções da gramática utilizadas:

- Statement = Designator ("=" Expr | ActPars) ";"
- Designator = ident { "." ident | "[" Expr "]" }
- Expr = ["-"] Term { Addop Term }
- Term = Factor { Mulop Factor }
- Factor = Designator [ActPars]
- Addop = "+" | "-"

Declaração 2 `total = arr[0] + arr[1];`

Produção inicial: Statement = Designator "=" Expr ";"

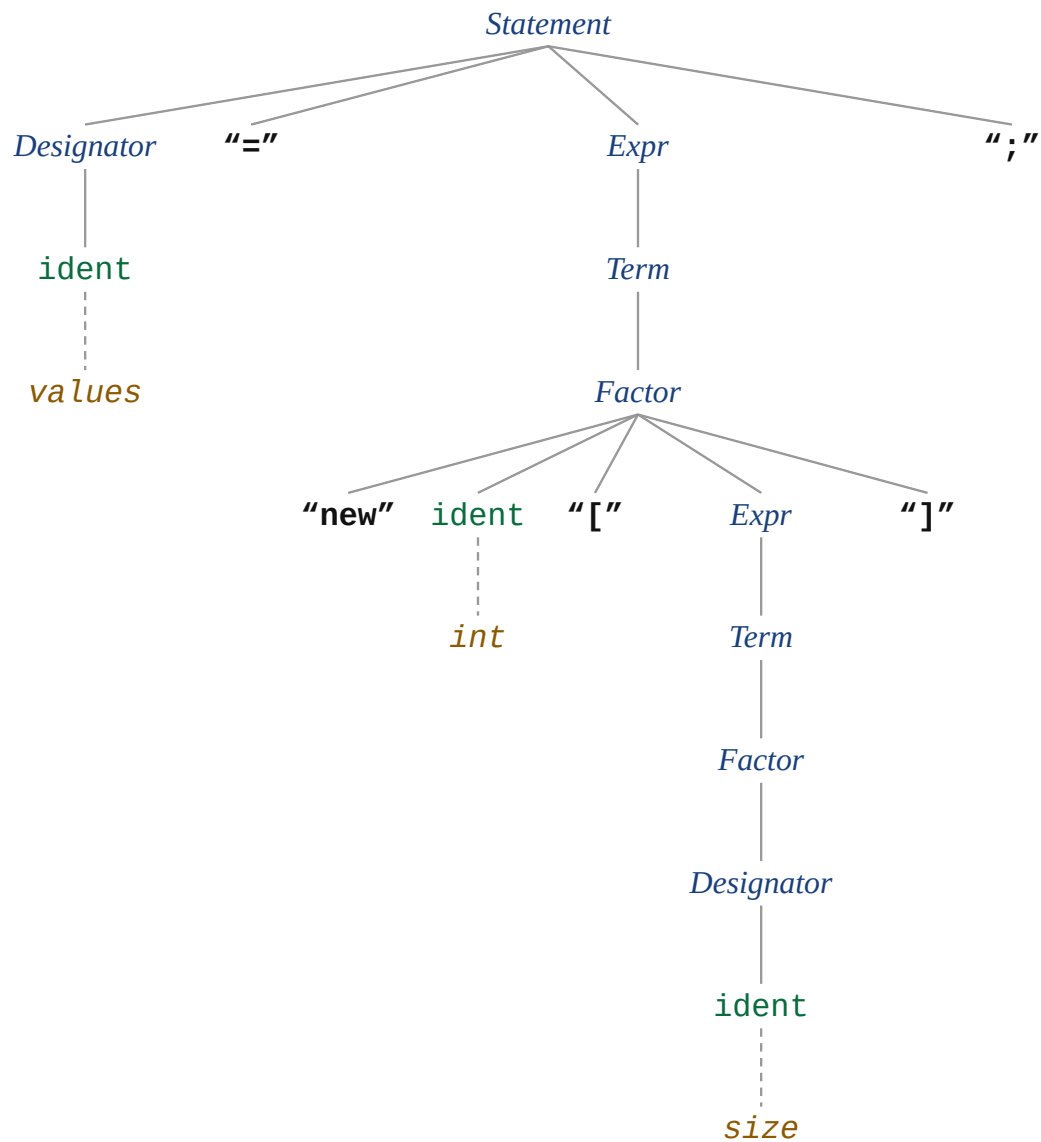


Produções da gramática utilizadas:

- Statement = Designator ("=" Expr | ActPars) ";"
- Designator = ident { "." ident | "[" Expr "]" }
- Expr = ["-"] Term { Addop Term }
- Term = Factor { Mulop Factor }
- Factor = Designator [ActPars] | number
- Addop = "+" | "-"

Declaração 3 `values = new int[size];`

Produção inicial: Statement = Designator "=" Expr ";"

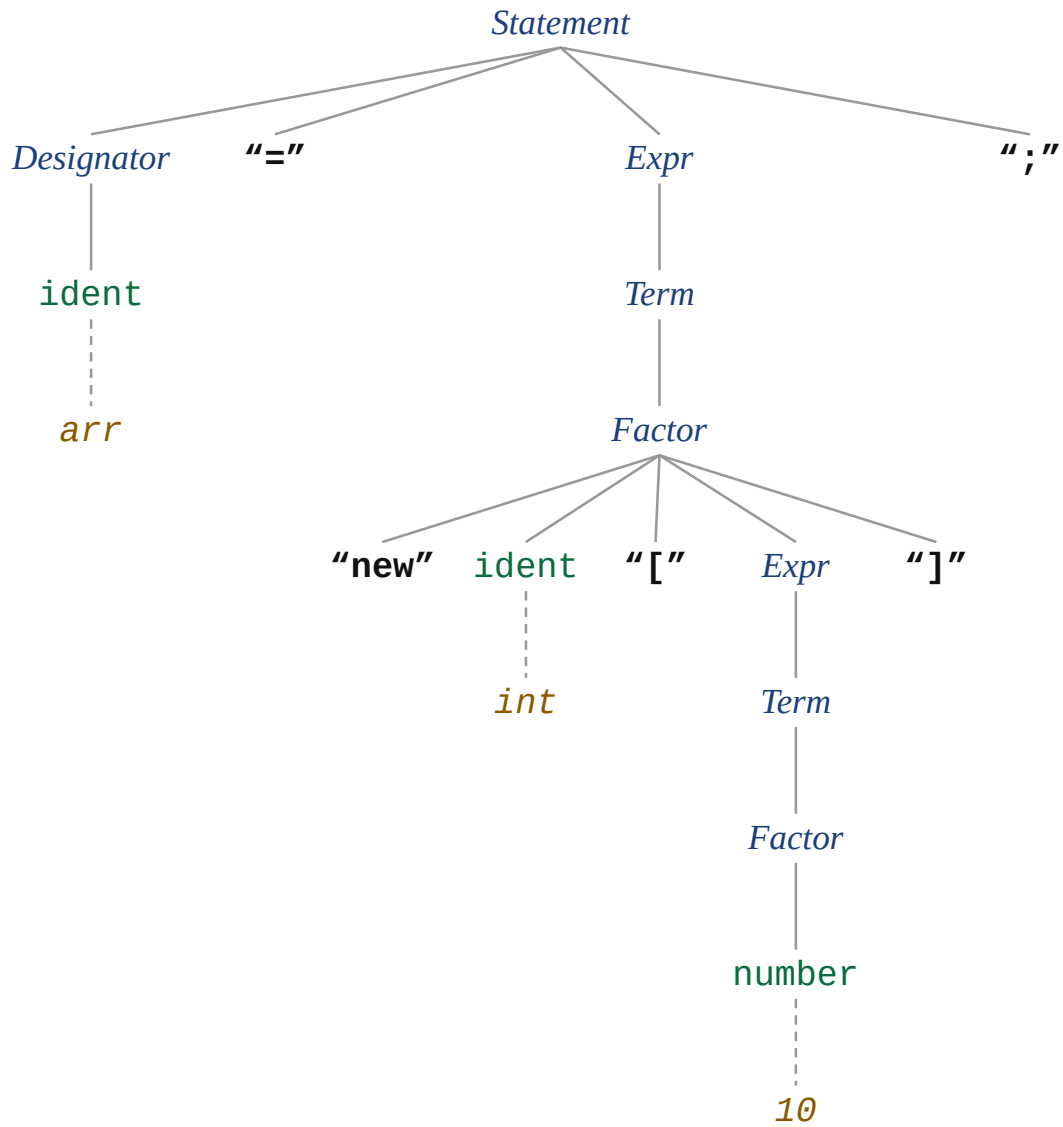


Produções da gramática utilizadas:

- Statement = Designator ("=" Expr | ActPars) ";"
- Designator = ident { "." ident | "[" Expr "]" }
- Expr = ["-"] Term { Addop Term }
- Term = Factor { Mulop Factor }
- Factor = "new" ident "[" Expr "]"

Declaração 4 `arr = new int[10];`

Produção inicial: Statement = Designator "=" Expr ";"

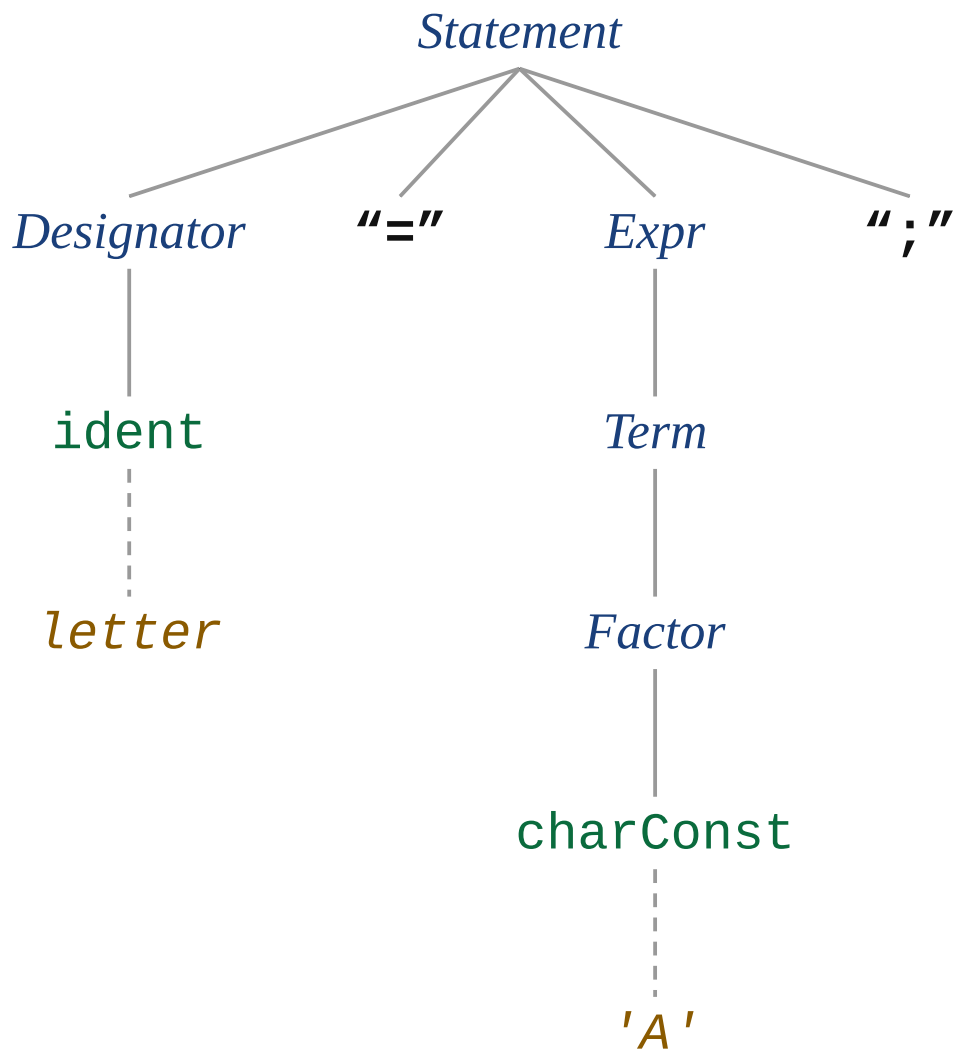


Produções da gramática utilizadas:

- Statement = Designator ("=" Expr | ActPars) ";"
- Designator = ident { "." ident | "[" Expr "]" }
- Expr = ["-"] Term { Addop Term }
- Term = Factor { Mulop Factor }
- Factor = "new" ident "[" Expr "]" | number

Declaração 5 `letter = 'A';`

Produção inicial: Statement = Designator "=" Expr ";"

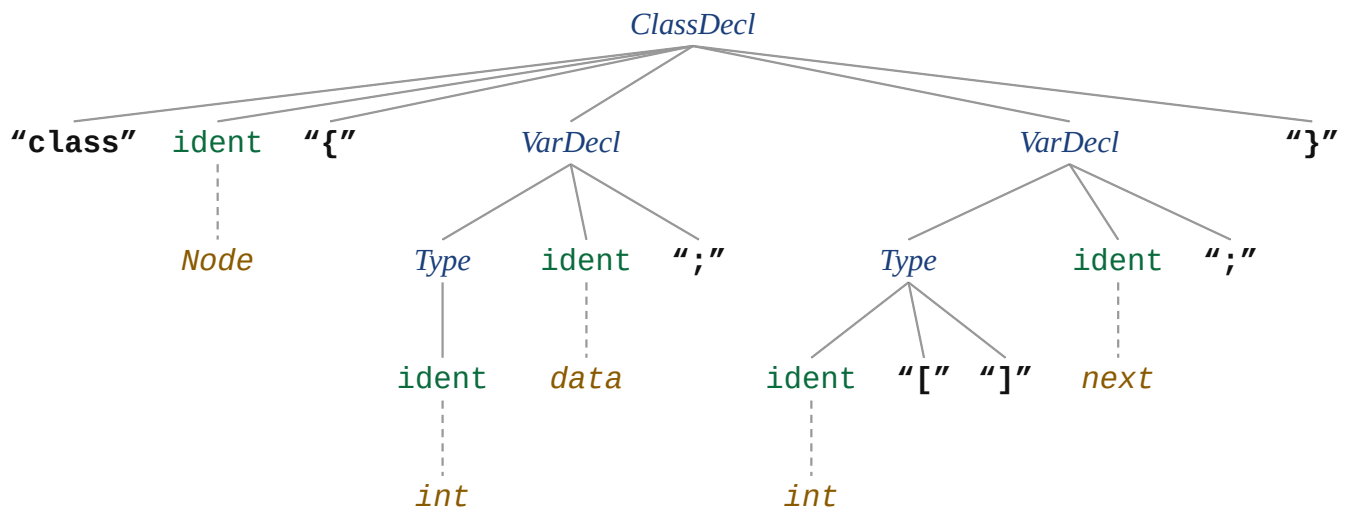


Produções da gramática utilizadas:

- Statement = Designator ("=" Expr | ActPars) ";"
- Designator = ident { "." ident | "[" Expr "]" }
- Expr = ["-"] Term { Addop Term }
- Term = Factor { Mulop Factor }
- Factor = charConst

Declaração 6 `class Node { int data; int[] next; }`

Produção inicial: `ClassDecl = "class" ident "{" {VarDecl} "}"`

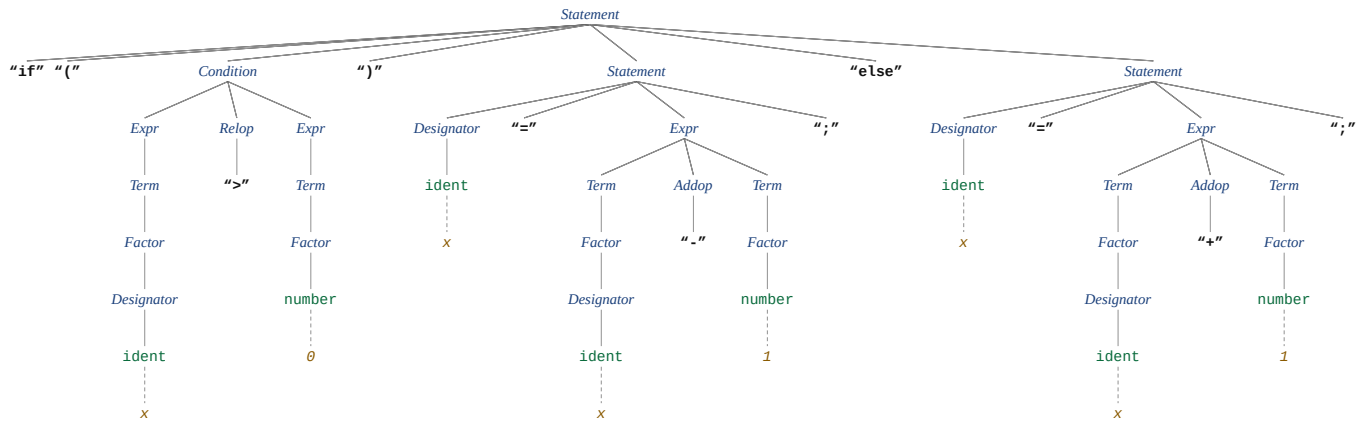


Produções da gramática utilizadas:

- `ClassDecl = "class" ident "{" {VarDecl} "}"`
- `VarDecl = Type ident {"," ident} ";"`
- `Type = ident "[" "]"`

Declaração 7 `if (x > 0) x = x - 1; else x = x + 1;`

Produção inicial: Statement = "if" "(" Condition ")" Statement ["else" Statement]

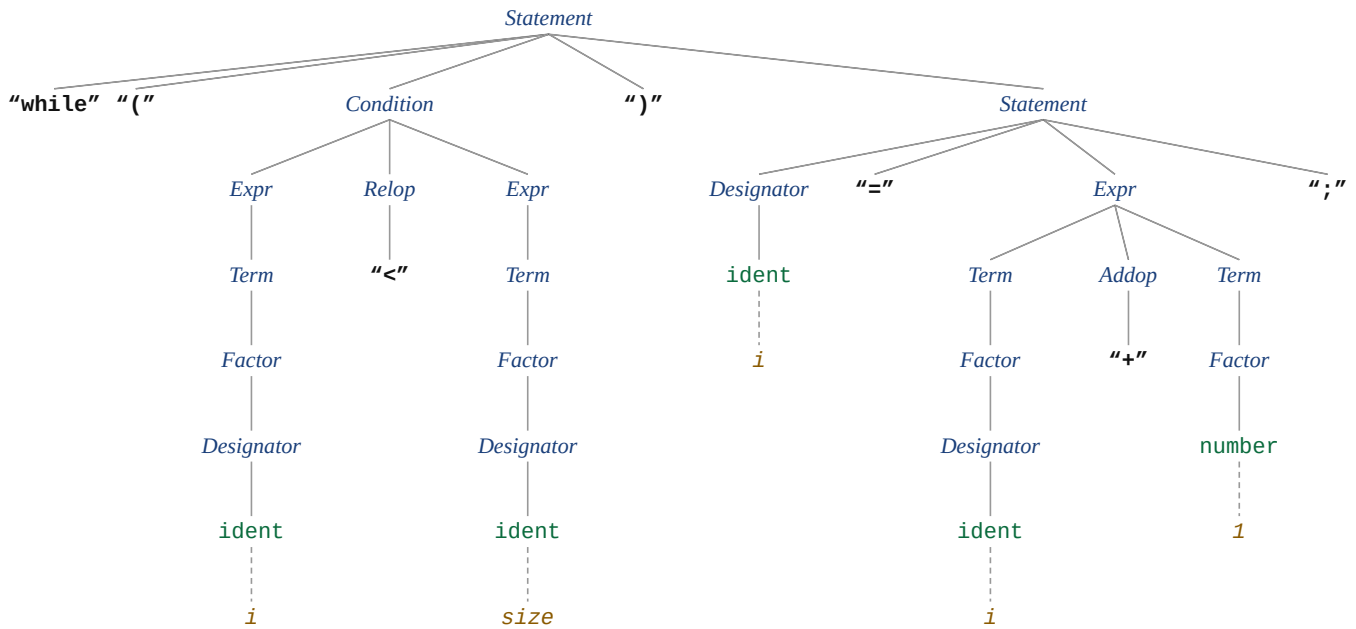


Produções da gramática utilizadas:

- Statement = "if" "(" Condition ")" Statement ["else" Statement]
- Statement = Designator ("=" Expr | ActPars) ";"
- Condition = Expr Relop Expr
- Relop = "==" | "!=" | ">" | ">=" | "<" | "<="
- Expr = ["-"] Term {Addop Term}
- Term = Factor {Mulop Factor}
- Factor = Designator [ActPars] | number
- Addop = "+" | "-"

Declaração 8 `while (i < size) i = i + 1;`

Produção inicial: Statement = "while" "(" Condition ")" Statement



Produções da gramática utilizadas:

- Statement = "while" "(" Condition ")" Statement
- Statement = Designator ("=" Expr | ActPars) ";"
- Condition = Expr Relop Expr
- Relop = "==" | "!=" | ">" | ">=" | "<" | "<="
- Expr = ["-"] Term {Addop Term}
- Term = Factor {Mulop Factor}
- Factor = Designator [ActPars] | number
- Addop = "+" | "-"